



Docker: Do Zero ao Compose

Bem-vindos! Este material servirá como seu guia prático e teórico para entender e operar o Docker. Como futuros profissionais de Redes de Computadores, entender a containerização é essencial, pois ela mudou drasticamente a forma como distribuimos serviços, roteamos portas e gerenciamos infraestruturas.

1. Conceitos Fundamentais

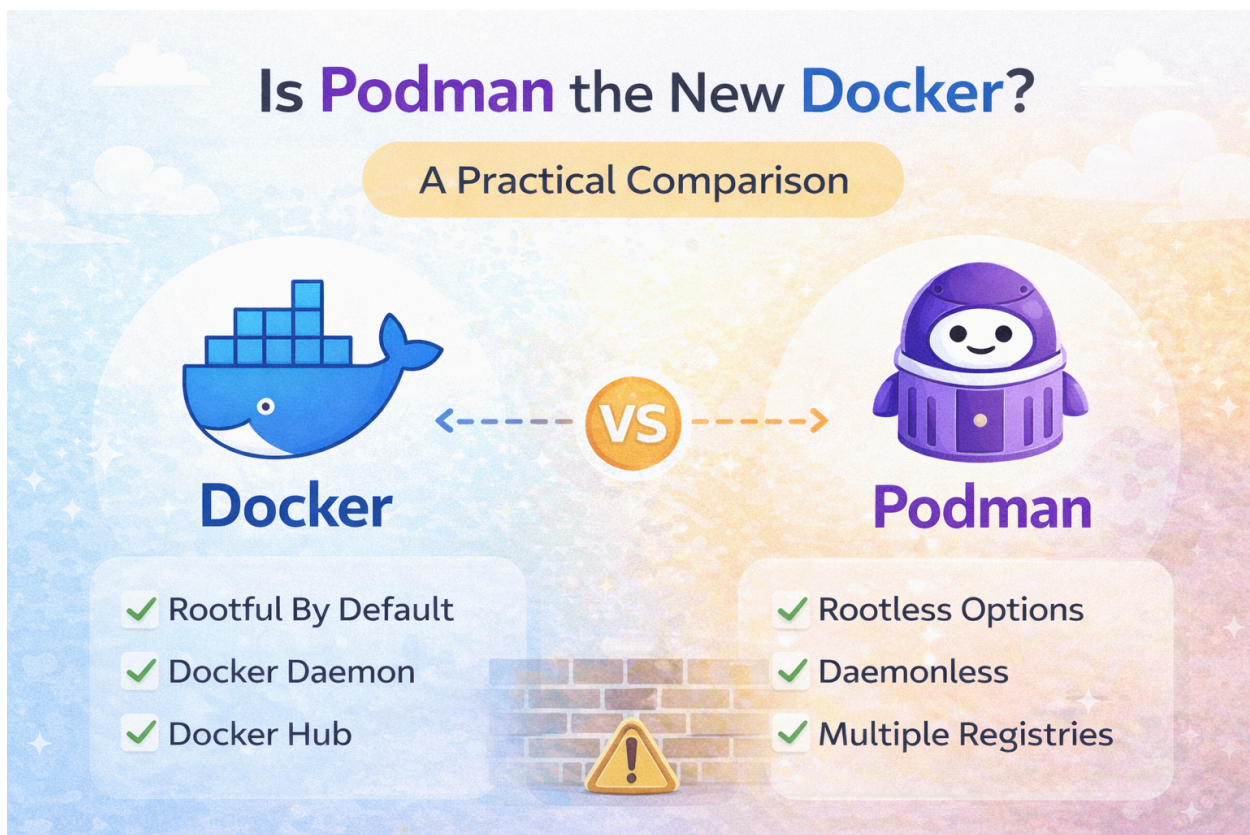
Antes de colocar a mão na massa, precisamos entender os dois pilares dessa tecnologia:

- **O que é o Docker?**

O Docker é uma plataforma de software que facilita a criação, o teste e a implantação de aplicativos. Ele empacota o software em unidades padronizadas chamadas de *containers*. Ao contrário das Máquinas Virtuais (VMs) tradicionais, que exigem um Sistema Operacional convidado completo para cada aplicação, o Docker compartilha o *kernel* (núcleo) do Sistema Operacional hospedeiro. Isso o torna extremamente leve, rápido e eficiente no uso de recursos.

- **O que é um Container?**

Um container é um ambiente isolado onde uma aplicação roda com todas as suas dependências (bibliotecas, arquivos de configuração, etc.). Para a área de redes, pense no container como um *host* virtual minúsculo: ele possui sua própria interface de rede, seu próprio endereço IP interno, sua própria tabela de roteamento e regras de firewall isoladas do sistema principal, mas consumindo uma fração mínima de memória e CPU.



2. Preparação do Ambiente: Configurando o Docker sem Sudo

Passo 1: Instalação do Docker Engine

Para garantir que teremos a versão mais atualizada e segura, é uma boa prática seguir a documentação oficial, pois os repositórios padrão do Debian/Ubuntu muitas vezes contêm versões desatualizadas do Docker.

Siga o passo a passo de instalação para Debian diretamente no link oficial:

<https://docs.docker.com/engine/install/debian/>

Dica após a instalação:

Para garantir que o serviço do Docker inicie automaticamente sempre que a máquina for reiniciada, execute o comando abaixo no terminal do Debian:

```
sudo systemctl enable --now docker
```

Passo 2: Rodar Docker sem sudo

Por padrão, o *daemon* (serviço) do Docker roda como usuário `root`. Para facilitar o uso no dia a dia e não precisarmos digitar `sudo` antes de cada comando, vamos configurar as permissões do seu usuário comum.

 **AVISO DE SEGURANÇA:** Adicionar um usuário ao grupo docker é equivalente a dar privilégios de root, pois o Docker permite montar diretórios do sistema host dentro de containers. Use apenas em ambientes de desenvolvimento pessoal.

Execute os comandos abaixo no seu terminal:

Bash

```
# 1. Crie o grupo docker (caso ainda não exista)
sudo groupadd docker

# 2. Adicione seu usuário ao grupo docker
sudo usermod -aG docker $USER
```

```
# 3. Ative as alterações de grupo (ou reinicie a máquina)
newgrp docker

# 4. Teste se o Docker roda sem o sudo
docker run hello-world
```

Se a mensagem "Hello from Docker!" aparecer, a configuração foi concluída com sucesso.



3. Módulo 1: Primeiros Passos e Docker Hub

- **O que é o Docker Hub?**

O Docker Hub é como a "App Store" do Docker. É de lá que baixamos as imagens oficiais para rodar nossos serviços.

Rodando o primeiro container

Bash

```
docker run hello-world
```

- **Explicação:** O Docker procura a imagem localmente. Como não acha, ele baixa do Docker Hub, cria o container, executa a mensagem e o container morre (encerra).

Listando Imagens Locais

Bash

```
docker images
```

- **Explicação:** Mostra o repositório, a tag (versão), o ID da imagem e o tamanho dos arquivos baixados na sua máquina.



4. Módulo 2: Ciclo de Vida e Gerenciamento de Containers

Rodar em Segundo Plano (Detached Mode)

Bash

```
docker run -d --name meu-nginx nginx
```

- **Explicação:** O parâmetro `d` libera o terminal. O parâmetro `-name` dá um nome amigável (se não colocar, o Docker inventa um nome aleatório).

Comandos de Gerenciamento

- **Listar containers em execução:** `docker ps`
- **Listar TODOS os containers (inclusive parados):** `docker ps -a`
- **Inspecionar os logs:** `docker logs meu-nginx`
- **Parar o container:** `docker stop meu-nginx` (Teste dar um `docker ps` depois disso. Ele sumirá, mas aparecerá no `docker ps -a` com status "Exited").
- **Remover um container específico:** `docker rm meu-nginx`

Acesso Interativo (Terminal do Container)

Bash

```
docker run -it ubuntu bash
```

- **Ação:** Você está dentro de um Ubuntu "limpo". Digite `exit` para sair.

💡 DICA DE OURO - Limpeza Total:

Para remover múltiplos containers de uma vez, use a expansão de comandos do Linux:

Bash

```
docker rm -f $(docker ps -aq)
```

- **Explicação:** O `f` força a parada e remoção. O comando `$(docker ps -aq)` lista apenas os IDs de todos os containers e os passa para o comando de

remoção.

5. Módulo 3: O Desafio do MySQL (Portas e Persistência)

Neste módulo, veremos a evolução de como rodar um banco de dados, enfrentando problemas comuns. Esta parte é **crucial para estudantes de redes**, pois lida diretamente com encaminhamento de portas (Port Forwarding/Binding).

Fase 1: Isolamento Total

Bash

```
docker run --name bd-teste -e MYSQL_ROOT_PASSWORD=senha123 -d mysql:8.0
```

- **Problema:** O banco está rodando, mas está isolado. Se você tentar conectar via MySQL Workbench na porta 3306 do IP da sua VM, vai dar erro. O Docker não expôs a porta para o mundo externo.
- *Limpeza:* Pare e remova o container antes de prosseguir (`docker stop bd-teste` e `docker rm bd-teste`).

Fase 2: Expondo a Porta (Port Binding)

Bash

```
docker run --name bd-teste -p 3306:3306 -e MYSQL_ROOT_PASSWORD=senha123 -d mysql:8.0
```

- **Explicação:** O parâmetro `-p <porta-da-vm>:<porta-do-container>` cria uma regra de NAT. Ele escuta a porta 3306 na sua máquina virtual e joga o tráfego para a porta 3306 dentro do container.
- **O Problema da Volatilidade:** Se você conectar no banco, criar um banco de dados chamado `loja_virtual`, destruir este container (`docker rm -f bd-teste`) e criá-lo novamente, o banco `loja_virtual` **sumiu!** Containers são efêmeros.

Fase 3: Armazenamento Persistente (Volumes)

Bash

```
docker run --name bd-teste -p 3306:3306 -v mysql-dados:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=senha123 -d mysql:8.0
```

- **Explicação:** O parâmetro `v <nome-do-volume>:<caminho-no-container>` resolve o problema. O Docker salvará os dados físicos fora do container, diretamente no disco da sua VM.
- **Prova de Fogo:** Se você criar o banco, destruir o container brutalmente e subi-lo novamente com o mesmo comando, os dados continuarão lá!

6. Módulo 4: Redes (Integrando Containers)

Como fazer dois containers conversarem entre si de forma isolada e segura, sem usar o IP da máquina? Usando as redes virtuais do Docker (Bridge Networks).

1. Criar uma rede virtual no Docker

Bash

```
docker network create minha-rede
```

2. Subir o MySQL dentro dessa rede

Bash

```
docker run --name meu-mysql --network minha-rede -e MYSQL_ROOT_PASSWORD=senha123 -d mysql:8.0
```

(Não precisamos do `-p` aqui, pois a comunicação será interna entre os containers).

3. Subir uma Aplicação Web (Adminer - Gerenciador de BD via Web)

Bash

```
docker run --name meu-adminer --network minha-rede -p 8080:8080 -d adminer
```

A Mágica do DNS Interno:

Acesse `http://<IP_DA_VM>:8080` no seu navegador. Na tela do Adminer, no campo "Servidor", **não digite o IP**. Digite apenas `meu-mysql` (o nome do container do banco)!

- **Contexto de Redes:** O Docker possui um servidor DNS embutido nas suas redes personalizadas. Ele resolve automaticamente o nome do container para o seu IP interno. Eles se acham pelo nome graças à rede criada!

(Limpe o ambiente para a próxima etapa usando a "Dica de Ouro" do Módulo 2).

7. Módulo 5: A Mágica do Docker Compose (O Grand Finale)

Digitar comandos gigantes no terminal é ruim e propenso a erros. O Compose resolve isso aplicando o conceito de **Infraestrutura como Código**, usando arquivos YAML.

1. Prepare o diretório do projeto:

Bash

```
mkdir meu-projeto && cd meu-projeto
```

2. Crie o arquivo de orquestração:

Bash

```
nano docker-compose.yml
```

3. Cole o conteúdo da infraestrutura (WordPress + MySQL):

YAML

```
version: '3.8'
services:
  db:
    image: mysql:8.0
    container_name: mysql_compose
    restart: always
```

```

environment:
  MYSQL_ROOT_PASSWORD: senha_secreta
  MYSQL_DATABASE: meu_app
ports:
  - "3306:3306"
volumes:
  - db_data:/var/lib/mysql

web:
  image: wordpress:latest
  container_name: wordpress_compose
  depends_on:
    - db
  ports:
    - "80:80"
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: root
    WORDPRESS_DB_PASSWORD: senha_secreta
    WORDPRESS_DB_NAME: meu_app
  volumes:
    - db_data:/var/www/html

volumes:
  db_data:

```

- **Services:** Define quais containers vamos subir (db e web).
- **Environment:** Variáveis de ambiente configurando o banco automaticamente e dizendo ao WordPress como se conectar (note o Host `db:3306`).
- **Volumes:** Criando a persistência no final do arquivo.

4. Subir toda a infraestrutura com um único comando:

Bash

```
docker compose up -d
```

(Teste acessando `http://<IP_DA_VM>` para ver a instalação do WordPress).

5. Derrubar tudo graciosamente:

Bash

```
docker compose down
```

- **Explicação final:** O `down` mata os containers e a rede virtual que ele criou, mas mantém o volume seguro para a próxima vez que você der um `up`.